03 Verilog Design

National Taiwan University of Science and Technology
Prof, Dr.-Ing Chi-Chia Sun
國立臺灣科技大學 電子工程系 宋啟嘉 教授

TAIWAN TECH

Overview Verilog HDL

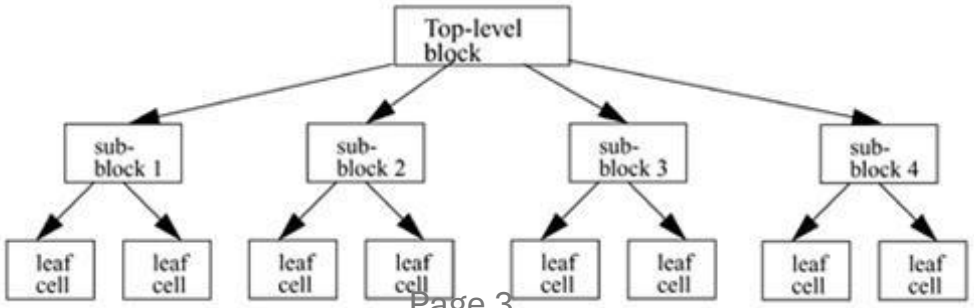
- Learning Objectives
  - *Top-down* vs. *Bottom-up* design methodologies
  - Explain differences between *Modules* and *module instances*
- Describe four levels of abstraction
  - *Behavior*, *data flow*, *gate level*, and *switch level*
- Describe components required for the simulation
  - *Stimulus block*
  - *Design block*

TAIWAN TECH

Design Methodologies

- Top-down
  - Define the top-level block
  - Identify the sub-blocks necessary to build the top level block
  - Further subdivide the sub-blocks until we come to leaf cells

Figure 2-1. Top-down Design Methodology

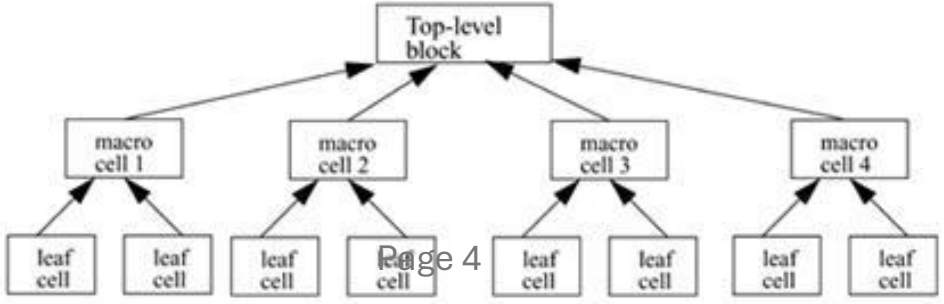


TAIWAN TECH

Design Methodologies

- Bottom-up
  - First identify the building blocks that are available to us
  - Build bigger cells using these building blocks
  - These cells are then used for higher-level blocks Until we build the top-level block in the design

Figure 2-2. Bottom-up Design Methodology

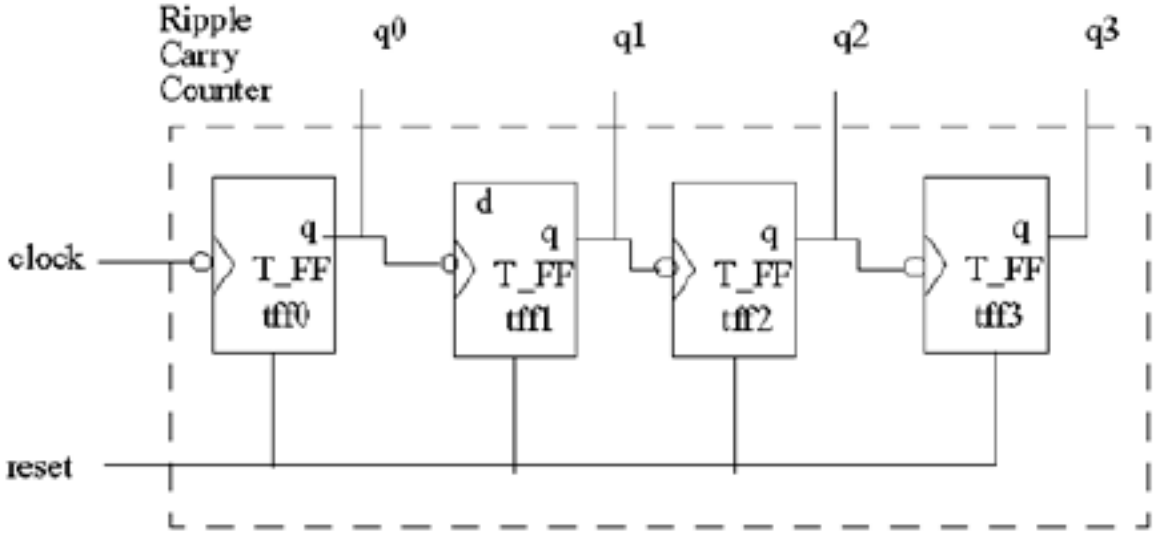


TAIWAN TECH

4-bit Ripple Carry Counter

- The ripple carry counter shown in Figure 2-3 is made up of negative edge-triggered toggle flipflops (*T_FF*).

Figure 2-3. Ripple Carry Counter



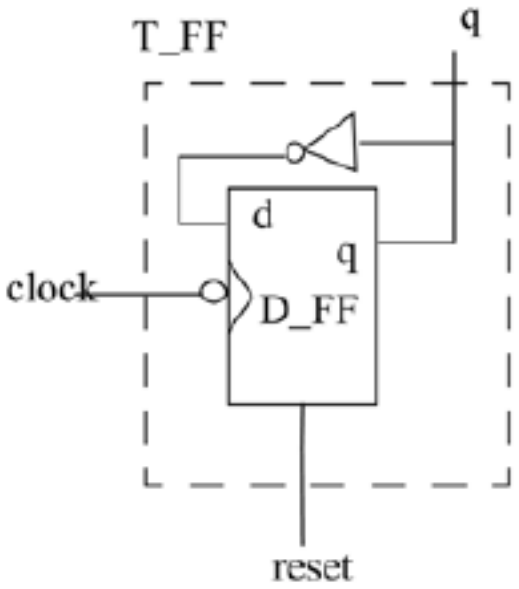
TAIWAN TECH

4-bit Ripple Carry Counter

- Each of the *T_FFs* can be made up from negative edge-triggered D-flipflops (*D_FF*) and *inverters*.
-

Figure 2-4. T-flipflop

reset	q_n	q_{n+1}
1	1	0
1	0	0
0	0	1
0	1	0
0	0	0

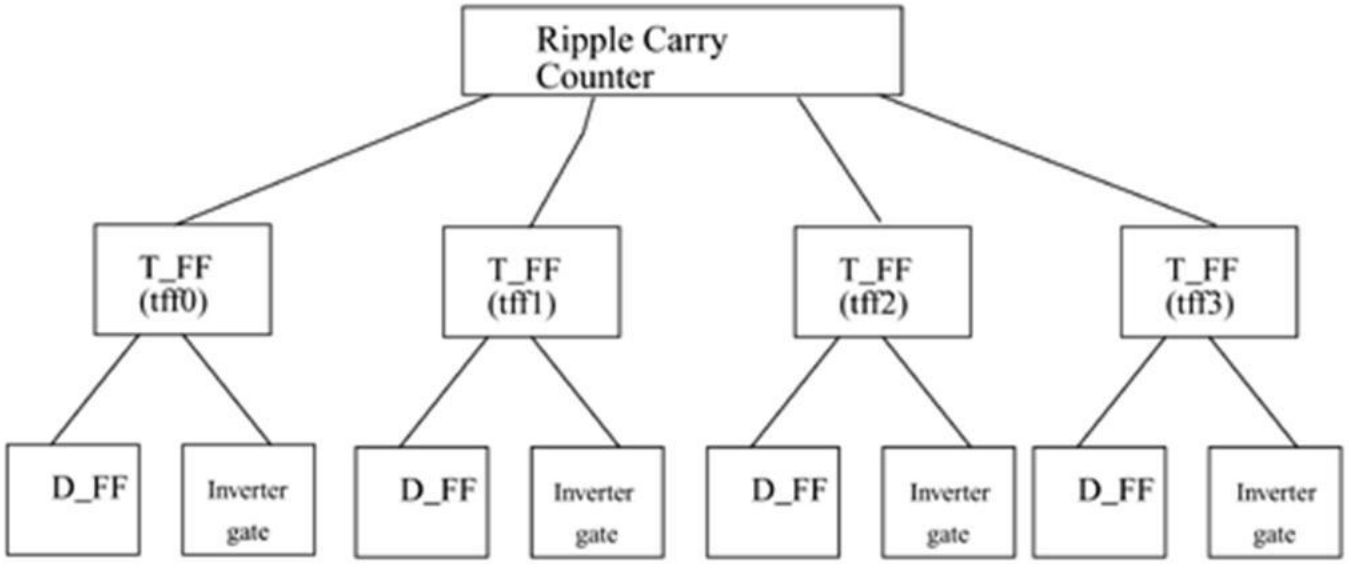


TAIWAN TECH

4-bit Ripple Carry Counter

- Thus, the ripple carry counter is built in a hierarchical fashion by using building blocks.

Figure 2-5. Design Hierarchy



TAIWAN TECH

Four different description levels

- Behavior or Algorithmic level
  - Consider function or algorithm of target
- Data flow level (RTL)
  - Assign data processing schemes
- Gate level
  - Using logic gates to construct circuits
- Switch level
  - Using transistors to construct circuits

TAIWAN TECH

Design Block

Example 2-4 Flipflop T_FF

```
module T_FF(q, clk, reset);

output q;
input clk, reset;
wire d;
D_FF dff0(q, d, clk, reset);
not n1(d, q); // not is a Verilog-provided primitive. case sensitive
endmodule
```

Example 2-5 Flipflop D_F

```
// module D_FF with synchronous reset
module D_FF(q, d, clk, reset);

output q;
input d, clk, reset;
reg q;

// Lots of new constructs.  Ignore the functionality of the
// constructs.
// Concentrate on how the design block is built in a top-down fashion.
always @(posedge reset or negedge clk)
if (reset)
    q <= 1'b0;
else
    q <= d;

endmodule
```


TAIWAN TECH

Design Block

Example 2-3 Ripple Carry Counter Top Block

```
module ripple_carry_counter(q, clk, reset);

output [3:0] q;
input clk, reset;

//4 instances of the module T_FF are created.
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);

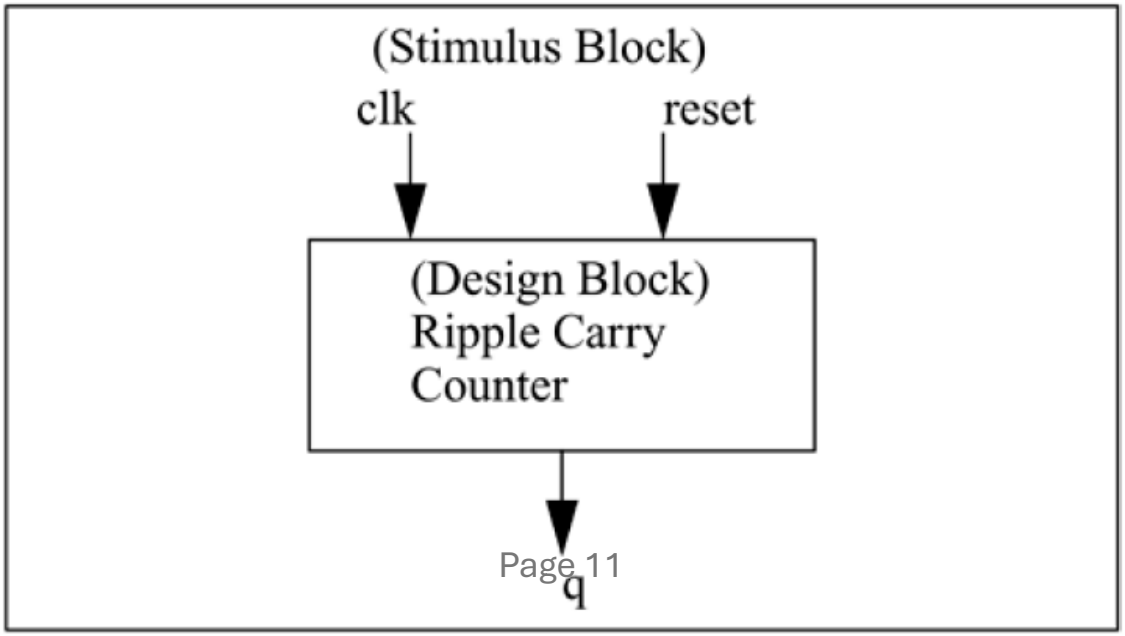
endmodule
```


TAIWAN TECH

Components of a Simulation

- Once a design block is completed,
  - *it must be tested*.
- Two styles of stimulus application are possible
  - Stimulus block instantiates design block

Figure 2-6. Stimulus Block Instantiates Design Block

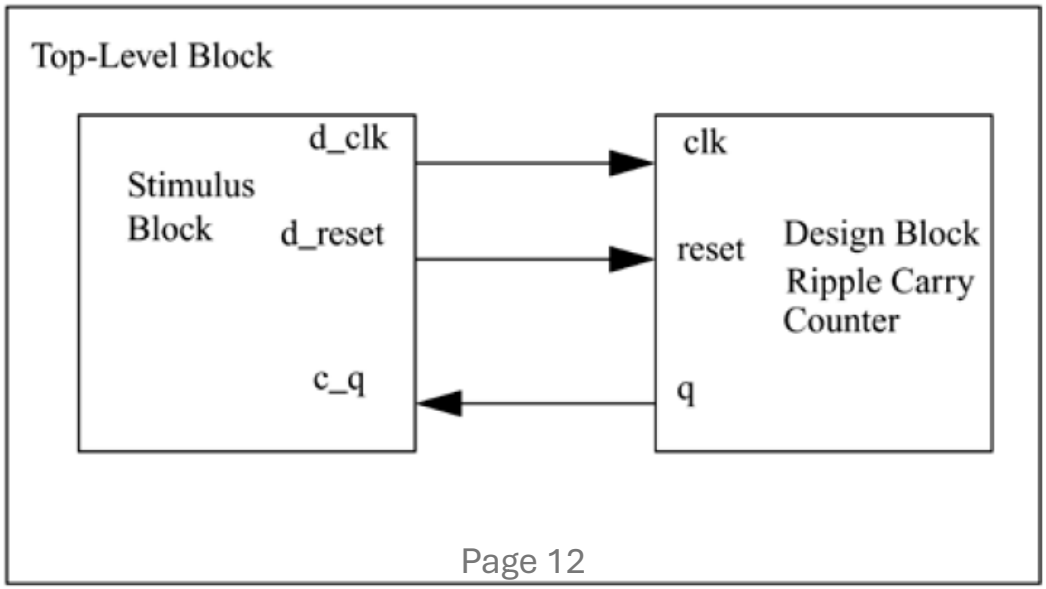


TAIWAN TECH

Components of a Simulation

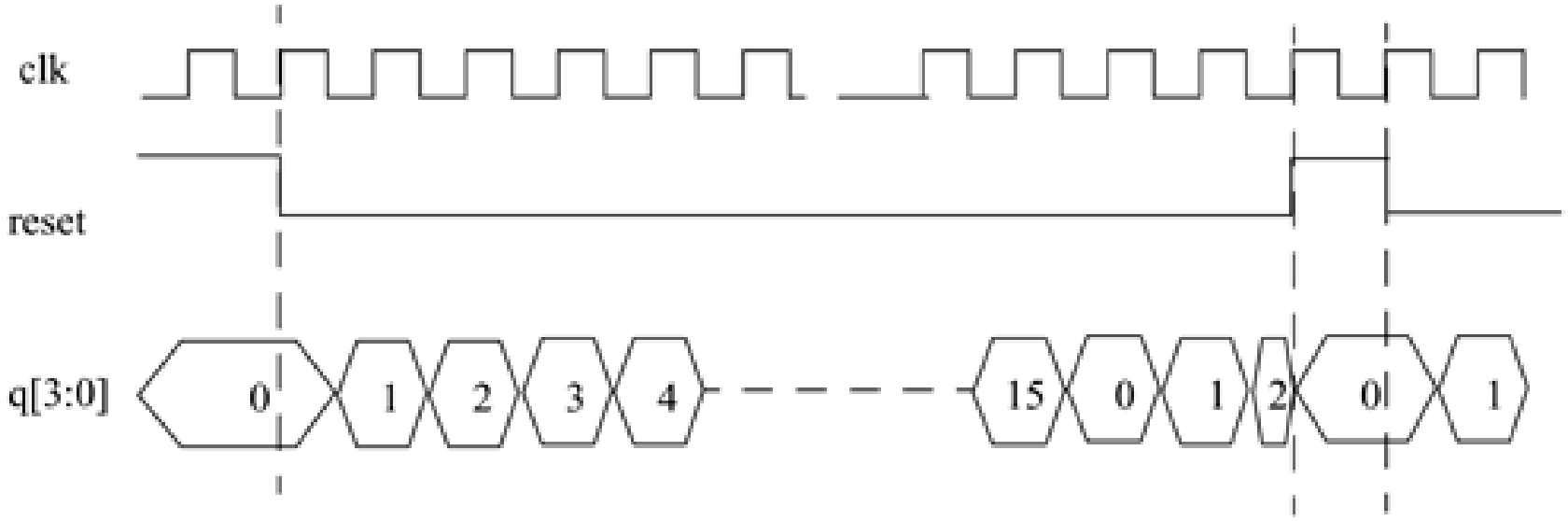
- Two styles of stimulus application are possible
  - Stimulus block instantiates design block
  - Stimulus and design blocks instantiated in a dummy top-level module

Figure 2-7. Stimulus and Design Blocks Instantiated in a Dummy Top-Level Module



Stimulus Block-Stimulus and Output Waveforms

Figure 2-8. Stimulus and Output Waveforms



Stimulus Block (first type)

```
module stimulus;

reg clk;
reg reset;
wire[3:0] q;

// instantiate the design block
ripple_carry_counter r1(q, clk, reset);

// Control the clk signal that drives the design block. Cycle time = 10
initial
   clk = 1'b0; //set clk to 0
always
   #5 clk = ~clk; //toggle clk every 5 time units

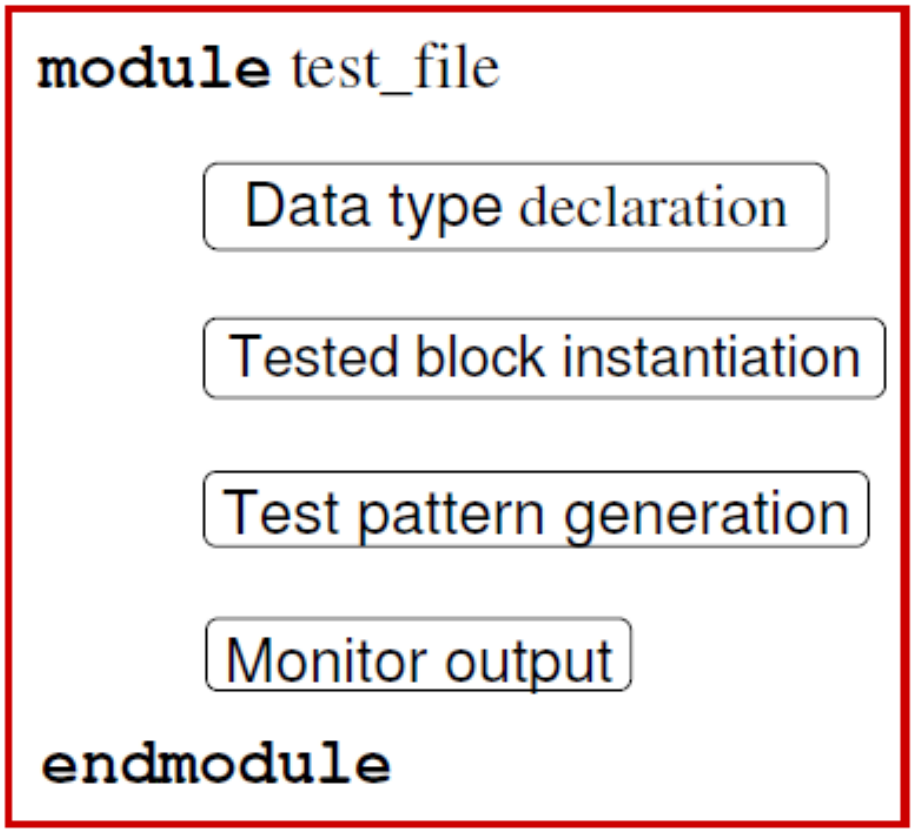
// Control the reset signal that drives the design block
// reset is asserted from 0 to 20 and from 200 to 220.
initial
begin
   reset = 1'b1;
   #15 reset = 1'b0;
   #180 reset = 1'b1;
   #10 reset = 1'b0;
   #20 $finish; //terminate the simulation
end

// Monitor the outputs
initial
   $monitor($time, " Output q = %d",  q);

endmodule
```


TAIWAN TECH

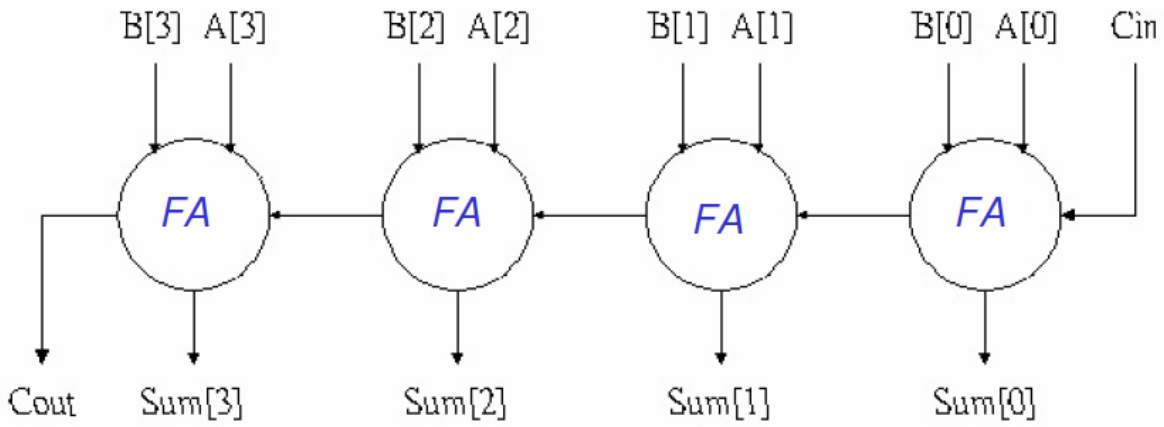
Stimulus Block (first type)



TAIWAN TECH

4-bits Ripple Carry Adder

Contains four 1-bit full adders (*FA*)



TAIWAN TECH

Lexical

- *Lexical conventions*
  - White space
  - Comments
  - Operators
  - Number specifications
  - Strings
  - Identifiers and keywords
  - Escaped identifiers
- Data types
- System tasks and compiler directives

TAIWAN TECH

Comments

- Comments
  - Single line – //
  - Multiple lines – /* . . . */
  - Can't be nested -- /* . . . /* . . . */ . . . */

```
/*
    2-to-1 multiplexer;
    out = a when sel = 0;
    out = b when sel = 1;
*/
module MUX_2(out, a, b, sel);
    output out;
    input a, b, sel;
    //netlist
    not N0(sel_, sel);
    and A0(a1, a, sel_),
        A1(b1b sel);
    or  O1(out,a1,a2);
endmodule
```


TAIWAN TECH

Operations

Arithmetic Operators	+, -, *, /, %
Relational Operators	<, <=, >, >=
Equality Operators	==, !=, ===, !==
Logical Operators	!, &&, ||
Bitwise and Unary Reduction Operators	~, &, |, ^, ~^
Shift Operators	>>, <<
Conditional Operators	?:
Concatenations	{ }
Replication	{{ }}

```
a = ~ b; // ~ is a unary operator. b is the operand
a = b && c; // && is a binary operator. b and c are operands
a = b ? c : d; // ?: is a ternary operator. b, c and d are operands
```


TAIWAN TECH

Number Representation

- Sized Number
  - binary, hex, decimal
  - 4bit, 12bits, 16bits

```
4'b1111 // This is a 4-bit   binary number
12'habc // This is a 12-bit  hexadecimal number
16'd255 // This is a 16-bit  decimal number.
```

- Unsized Number
  - without lengths

```
23456 // This is a 32-bit   decimal number by default
'hc3 // This is a 32-bit   hexadecimal number
'o21 // This is a 32-bit   octal number
```

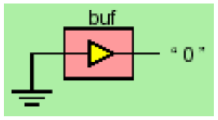
- Underscore character is allowed

```
12'b1111_0000_1010 // Use of underline characters for readability
4'b10?? // Equivalent of a 4'b10zz
```

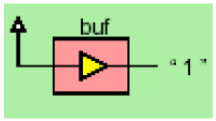

TAIWAN TECH

Data Type

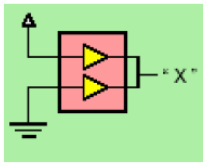
- 4 value level
  - 0, 1 standard digits
  - X, unknown or conflicts
  - Z, for bus connection



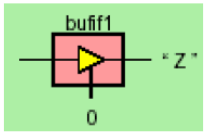
0:logic 0 / false



1:logic 1 / true



X:unknown logic value



Z:high-impedance

TAIWAN TECH

Nets

- Connections between HW elements
- Default value is Z
- Types (Keywords)
  - **wire (default) , tri**
  - **wand, triand**
  - **wor, trior**
  - **trireg, tri1, tri0**
- One-bit values by default

Figure 3-1. Example of Nets




```
wire a; // Declare net a for the above circuit
wire b,c; // Declare two wires b,c for the above circuit
wire d = 1'b0; // Net d is fixed to logic value 0 at declaration.
```


TAIWAN TECH

Registers

- Represent data storage elements
  - Declared by the keyword Reg
  - Default value is X
- Usually flip-flops
- With Clock as synchronous, become Latch

```
reg reset; // declare a variable reset that can hold its value
initial // this construct will be discussed later
begin
  reset = 1'b1; //initialize reset to 1 to reset the digital circuit.
  #100 reset = 1'b0; // after 100 time units reset is deasserted.
end
```

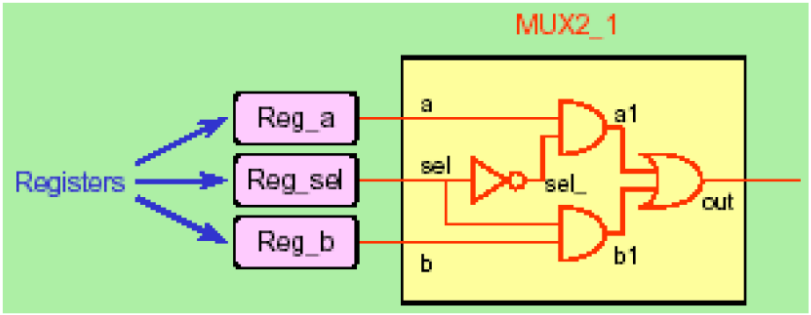
```
reg signed [63:0] m; // 64 bit signed value
integer i; // 32 bit signed value
```


TAIWAN TECH

Registers

- Register (flip-flop, latch, SDF)
- Integer (for-loop)
- Real (float)
- Time (simulation test)

Register Types	Function
`reg`	*Unsigned* integer variable of varying bit width
`integer`	*Signed* integer variable, 32-bit wide. Arithmetic operations produce 2's complement result.
`real`	*Signed* floating-point variable, double precision.
`time`	*Unsigned* integer variable, 64-bit wide (Verilog- XL stores simulation time as a 64-bit positive value).



TAIWAN TECH

Vectors

- Vectors can be declared as
  - [high# : low#] or [low# : high#]

```
reg [255:0] data1; //Little endian notation
reg [0:255] data2; //Big endian notation
reg [7:0] byte;

//Using a variable part select, one can choose parts

byte = data1[31-:8]; //starting bit = 31, width =8 => data[31:24]
byte = data1[24+:8]; //starting bit = 24, width =8 => data[31:24]
byte = data2[31-:8]; //starting bit = 31, width =8 => data[24:31]
byte = data2[24+:8]; //starting bit = 24, width =8 => data[24:31]

//The starting bit can also be a variable. The width has
//to be constant. Therefore, one can use the variable part select
//in a loop to select all bytes of the vector.
for (j=0; j<=31; j=j+1)
   byte = data1[(j*8)+:8]; //Sequence is [7:0], [15:8]... [255:248]

//Can initialize a part of the vector
data1[(byteNum*8)+:8] = 8'b0; //If byteNum = 1, clear 8 bits [15:8]
```


TAIWAN TECH

Arrays

- Allowed for **reg**, **wire**, **integer**, **time, real, realtime,** and vector register data types
- 2D vectors for Memory
  - Similar to c array

```
integer count[0:7]; // An array of 8 count variables

reg bool[31:0]; // Array of 32 one-bit boolean register variables

time chk_point[1:100]; // Array of 100 time checkpoint variables

reg [4:0] port_id[0:7]; // Array of 8 port_ids; each port_id is 5 bits wide

integer matrix[4:0][0:255]; // Two dimensional array of integers

reg [63:0] array_4d [15:0][7:0][7:0][255:0]; //Four dimensional array

wire [7:0] w_array2 [5:0]; // Declare an array of 8 bit vector wire

wire w_array1[7:0][5:0]; // Declare an array of single bit wires
```


TAIWAN TECH

Arrays

- How to access the array elements
  - regs[3]=b, write
  - x=regs[3], read

Examples of assignments to elements of arrays discussed above are shown below:

```
count[5] = 0; // Reset 5th element of array of count variables
chk_point[100] = 0; // Reset 100th time check point value
port_id[3] = 0; // Reset 3rd element (a 5-bit value) of port_id array.

matrix[1][0] = 33559; // Set value of element indexed by [1][0] to 33559
array_4d[0][0][0][0][15:0] = 0; //Clear bits 15:0 of the register
                               //accessed by indices [0][0][0][0]

port_id = 0; // Illegal syntax - Attempt to write the entire array
matrix [1] = 0; // Illegal syntax - Attempt to write [1][0]..[1][255]
```


TAIWAN TECH

Arrays (Memory)

- Models simply as an array of registers
  - Each element of the array is known as a word
    - Each word can be one or more bits
    - n 1-bit registers vs. 1 n-bit register

```
reg mem1bit[0:1023];        //1k 1-bit words
reg [7:0] membyte[0:1023]; //1k 8-bit words

membyte[511]  //fetches 1 byte word at 511
```


TAIWAN TECH

System calls

- $monitor(p1,p2,p3,…pn)
  - Useful function call during the simulation for debug

```
//Monitor time and value of the signals clock and reset
//Clock toggles every 5 time units and reset goes down at 10 time units
initial
begin
    $monitor($time,
             " Value of signals clock = %b reset = %b", clock,reset);
end
```

Partial output of the monitor statement:

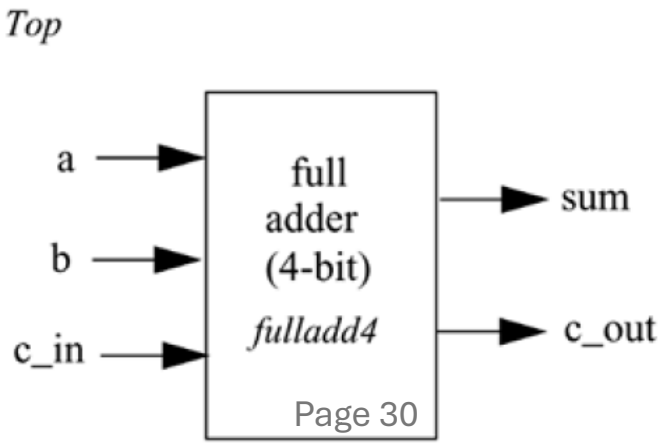
```
-- 0 Value of signals clock = 0 reset = 1
-- 5 Value of signals clock = 1 reset = 1
-- 10 Value of signals clock = 0 reset = 0
```


TAIWAN TECH

Ports

- Provide the interface A module can communicate with its environment
  - I/O pins of an IC chip are its ports
  - Provides a very powerful flexibility to the designer

```
module fulladd4(sum, c_out, a, b, c_in); //Module with a list of ports
module Top; // No list of ports, top-level module in simulation
```



TAIWAN TECH

Ports

- Ports in the list of ports must be declared
  - **input => input port**
  - **output => output port**
  - **inout => bidirectional port**

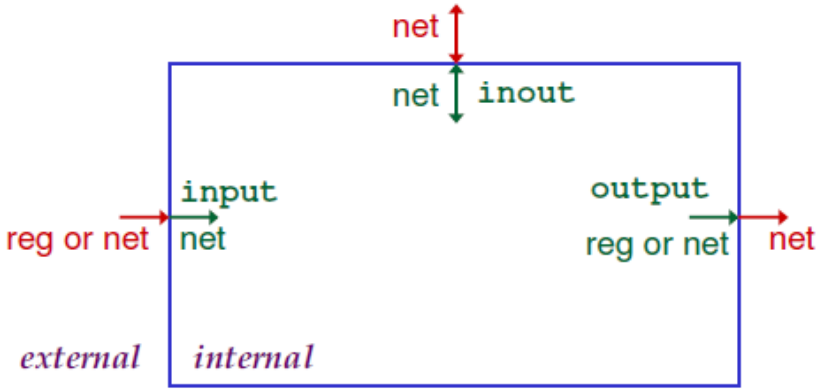
```
module fulladd4(sum, c_out, a, b, c_in);

//Begin port declarations section
output[3:0] sum;
output c_cout;
input [3:0] a, b;
input c_in;
//End port declarations section
...
<module internals>
...
endmodule
```


TAIWAN TECH

Ports

- **Input**
  - Source: **reg or net**
  - Destination: **net**
- **Output**
  - Source: **reg or net**
  - Destination: **net**
- **Inout**
  - Source: **net**
  - Destination: **net**



TAIWAN TECH

Ports

- Illegal define

```
module Top;

//Declare connection variables
reg [3:0]A,B;
reg C_IN;
reg [3:0] SUM;
wire C_OUT;

    //Instantiate fulladd4, call it fa0
    fulladd4 fa0(SUM, C_OUT, A, B, C_IN);
    //Illegal connection because output port sum in module fulladd4
    //is connected to a register variable SUM in module Top.
    .
    .
    <stimulus>
    .
    .
endmodule
```


TAIWAN TECH

Models with ports

- Connect the port by name
  - Important for many many Ios

```
// Instantiate module fa_byname and connect signals to ports by name
fulladd4 fa_byname(.c_out(C_OUT), .sum(SUM), .b(B), .c_in(C_IN), .a(A),);
```

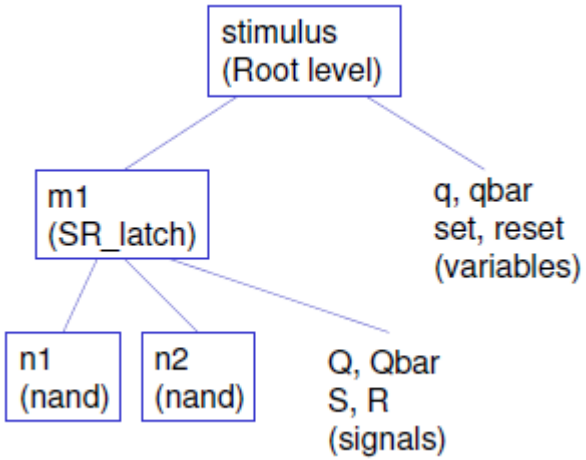
- Connect the port by order
  - Fine for adder or small design

```
//Instantiate fulladd4, call it fa_ordered.
//Signals are connected to ports in order (by position)
fulladd4 fa_ordered(SUM, C_OUT, A, B, C_IN);
```


TAIWAN TECH

Hierarchy Names

- Design Hierarchy for SR Latch Simulation




```
stimulus
stimulus.qbar
stimulus.reset
stimulus.m1.Q
stimulus.m1.S
stimulus.n1
```

```
stimulus.q
 stimulus.set
 stimulus.m1
 stimulus.m1.Qbar
 stimulus.m1.R
stimulus.n2
```


TAIWAN TECH

Operands

Operands can be any one of the data types defined in Section 3.2, *Data Types*
Some constructs will take only certain types of operands. Operands can be *constants, integers, real numbers, nets, registers, times, bit-select* (one bit of vector net or a vector register), *part-select* (selected bits of the vector net or register vector), and *memories* or *function calls* (functions are discussed later)

```
integer count, final_count;
final_count = count + 1;//count is an integer operand

real a, b, c;
c = a - b; //a and b are real operands

reg [15:0] reg1, reg2;
reg [3:0] reg_out;
reg_out = reg1[3:0] ^ reg2[3:0];//reg1[3:0] and reg2[3:0] are
                                //part-select register operands

reg ret_value;
ret_value = calculate_parity(A, B);//calculate_parity is a
                                   //function type operand
```


TAIWAN TECH

Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	*	multiply	2
	/	divide	2
	+	add	2
	-	subtract	2
	%	modulus	2
	**	power (exponent)	2
Logical	!	logical negation	1
	&&	logical and	2
	||	logical or	2
Relational	>	greater than	2
	<	less than	2
	>=	greater than or equal	2
	<=	less than or equal	2
Equality	==	equality	2
	!=	inequality	2
	===	case equality	2
	!==	case inequality	2

TAIWAN TECH

Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Bitwise	~	bitwise negation	1
	&	bitwise and	2
	|	bitwise or	2
	^	bitwise xor	2
	^~ or ~^	bitwise xnor	2
Reduction	&	reduction and	1
	~&	reduction nand	1
	|	reduction or	1
	~|	reduction nor	1
	^	reduction xor	1
	^~ or ~^	reduction xnor	1
Shift	>>	Right shift	2
	<<	Left shift	2
	>>>	Arithmetic right shift	2
	<<<	Arithmetic left shift	2
Concatenation	{ }	Concatenation	Any number
Replication	{ { } }	Replication	Any number
Conditional	?:	Conditional	3

TAIWAN TECH

Arithemtic operators

```
A = 4'b0011; B = 4'b0100; // A and B are register vectors
D = 6; E = 4; F=2// D and E are integers

A * B // Multiply A and B. Evaluates to 4'b1100
D / E // Divide D by E. Evaluates to 1. Truncates any fractional part.
A + B // Add A and B. Evaluates to 4'b0111
B - A // Subtract A from B. Evaluates to 4'b0001
F = E ** F; //E to the power F, yields 16
```

```
13 % 3 // Evaluates to 1
16 % 4 // Evaluates to 0
-7 % 2 // Evaluates to -1, takes sign of the first operand
7 % -2 // Evaluates to +1, takes sign of the first operand
```


TAIWAN TECH

If any operand bit has a value x

- in1 = 4'b101x;
- in2 = 4'b1010;
- sum = in1 + in2;
  - sum will be evaluated to the value 4'bxxxx

TAIWAN TECH

Logic Operators

- 1 is True, and 0 is False

```
// Logical operations
A = 3; B = 0;
A && B // Evaluates to 0. Equivalent to (logical-1 && logical-0)
A || B // Evaluates to 1. Equivalent to (logical-1 || logical-0)
!A// Evaluates to 0. Equivalent to not(logical-1)
!B// Evaluates to 1. Equivalent to not(logical-0)

// Unknowns
A = 2'b0x; B = 2'b10;
A && B // Evaluates to x. Equivalent to (x && logical 1)

// Expressions
(a == 2) && (b == 3) // Evaluates to 1 if both a == 2 and b == 3 are true.
 // Evaluates to 0 if either is false.
```


TAIWAN TECH

Rational Operators

- <=, >, >= and <
  - Same to c, 1 is true, and 0 is false
  - When unknown occurs, return x

```
// A = 4, B = 3
// X = 4'b1010, Y = 4'b1101, Z = 4'b1xxx

A <= B // Evaluates to a logical 0
A > B // Evaluates to a logical 1
Y >= X // Evaluates to a logical 1
Y < Z // Evaluates to an x
```


TAIWAN TECH

Equality Operators

Expression	Description	Possible Logical Value
a == b	a equal to b, result unknown if **x** or **z** in a or b	**0, 1, x**
a != b	a not equal to b, result unknown if **x** or **z** in a or b	**0, 1, x**
a === b	a equal to b, including **x** and **z**	**0, 1**
a !== b	a not equal to b, including **x** and **z**	**0, 1**

```
// A = 4, B = 3
// X = 4'b1010, Y = 4'b1101
// Z = 4'b1xxz, M = 4'b1xxz, N = 4'b1xxx

A == B // Results in logical 0
X != Y // Results in logical 1
X == Z // Results in x
Z === M // Results in logical 1 (all bits match, including x and z)
Z === N // Results in logical 0 (least significant bit does not match)
M !== N // Results in logical 1
```


TAIWAN TECH

Bitwise Operators

- Bitwise operators are
  - negation (~), and(&), or (|)
  - xor (^), xnor (^~, ~^).

bitwise and	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

bitwise or	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

bitwise xor	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

bitwise xnor	0	1	x
0	1	0	x
1	0	1	x
x	x	x	x

bitwise negation	result
0	1
1	0
x	x

TAIWAN TECH

Logical operators perform a logical operation

```
// X = 4'b1010, Y = 4'b1101
// Z = 4'b10x1

~X     // Negation. Result is 4'b0101
X & Y  // Bitwise and. Result is 4'b1000
X | Y  // Bitwise or. Result is 4'b1111
X ^ Y  // Bitwise xor. Result is 4'b0111
X ^~ Y // Bitwise xnor. Result is 4'b1000
X & Z  // Result is 4'b10x0
```

```
// X = 4'b1010, Y = 4'b0000

X | Y // bitwise operation. Result is 4'b1010
X || Y // logical operation. Equivalent to 1 || 0. Result is 1.
```


TAIWAN TECH

Shift Operators

- Shift operators are right shift (>>), left shift (<<), arithmetic right shift (>>>), and arithmetic left shift (<<<).

```
// X = 4'b1100

Y = X >> 1; //Y is 4'b0110. Shift right 1 bit. 0 filled in MSB position.
Y = X << 1; //Y is 4'b1000. Shift left 1 bit. 0 filled in LSB position.
Y = X << 2; //Y is 4'b0000. Shift left 2 bits.

integer a, b, c; //Signed data types
a = 0;
b = -10; // 00111...10110 binary
c = a + (b >>> 3); //Results in -2 decimal, due to arithmetic shift
```


TAIWAN TECH

Concatenation Operator

```
// A = 1'b1, B = 2'b00, C = 2'b10, D = 3'b110

Y = {B , C} // Result Y is 4'b0010
Y = {A , B , C , D , 3'b001} // Result Y is 11'b10010110001
Y = {A , B[0], C[1]} // Result Y is 3'b101
```


TAIWAN TECH

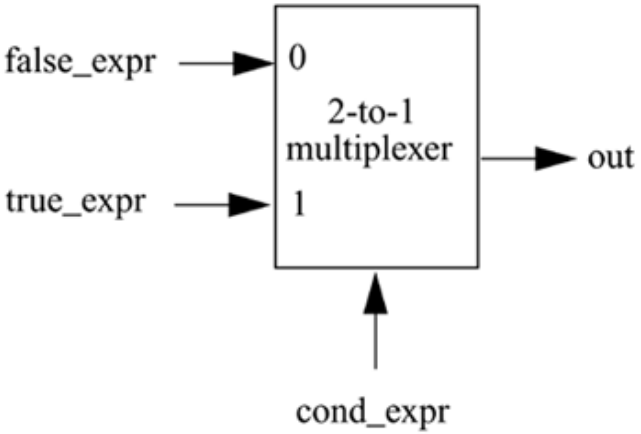
Replication Operator

```
reg A;
reg [1:0] B, C;
reg [2:0] D;
A = 1'b1; B = 2'b00; C = 2'b10; D = 3'b110;

Y = { 4{A} } // Result Y is 4'b1111
Y = { 4{A} , 2{B} } // Result Y is 8'b11110000
Y = { 4{A} , 2{B} , C } // Result Y is 8'b1111000010
```


TAIWAN TECH

Conditional Operator



Conditional operators are frequently used in dataflow modeling to model conditional assignments. The conditional expression acts as a switching control.

```
//model functionality of a tristate buffer
assign addr_bus = drive_enable ? addr_out : 36'bz;

//model functionality of a 2-to-1 mux
assign out = control ? in1 : in0;
```

Conditional operations can be nested. Each *true_expr* or *false_expr* can itself be a conditional operation. In the example that follows, convince yourself that (*A==3*) and control are the two select signals of 4-to-1 multiplexer with *n, m, y, x* as the inputs and *out* as the output signal.

```
assign out = (A == 3) ? ( control ? x : y ): ( control ? m : n) ;
```


TAIWAN TECH

Operator Precedence

Operators	Operator Symbols	Precedence
Unary	+ - ! ~	Highest precedence
Multiply, Divide, Modulus	* / %	
Add, Subtract	+ -	
Shift	<< >>	
Relational	< <= > >=	
Equality	== != === !==	
Reduction	&, ~& ^ ^~ |, ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

TAIWAN TECH

4-to-1 Multiplexer, Using Logic Equations

Example 6-2 4-to-1 Multiplexer, Using Logic Equations

```
// Module 4-to-1 multiplexer using data flow. logic equation
// Compare to gate-level model
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);

// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;

//Logic equation for out
assign out =    (~s1 & ~s0 & i0)|
                (~s1 & s0 & i1) |
                (s1 & ~s0 & i2) |
                (s1 & s0 & i3) ;

endmodule
```


TAIWAN TECH

4-to-1 Multiplexer, Using Conditional Operators

```
// Module 4-to-1 multiplexer using data flow. Conditional operator.
// Compare to gate-level model
module multiplexer4_to_1 (out, i0, i1, i2, i3, s1, s0);

// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;

// Use nested conditional operator
assign out = s1 ? ( s0 ? i3 : i2) : (s0 ? i1 : i0) ;

endmodule
```


TAIWAN TECH

4-bit Full Adder, Using Dataflow Operators

```
// Define a 4-bit full adder by using dataflow statements.
module fulladd4(sum, c_out, a, b, c_in);

// I/O port declarations
output [3:0] sum;
output c_out;
input[3:0] a, b;
input c_in;

// Specify the function of a full adder
assign {c_out, sum} = a + b + c_in;

endmodule
```


TAIWAN TECH

Structured Procedures

- Two basic structured procedure statements
  - `Always`
  - `Initial`
- All behavioral statements can appear only inside these blocks
- Each always or initial block has a separate activity flow (concurrency)
- Start from simulation time 0
- Cannot be nested

TAIWAN TECH

Structured Procedures: initial statement

- Starts at time 0
- Executes only once during a simulation
- Multiple `initial` blocks, execute in parallel
  - All start at time 0
  - Each finishes independently
- Syntax:

```
initial
begin
  // behavioral statements
end
```


TAIWAN TECH

Structured Procedures: initial statement (cont'd)

- Example:

```
module stimulus;
  reg x, y, a, b, m;

  initial
    m= 1'b0;

  initial
  begin
    #5 a=1'b1;
    #25 b=1'b0;
  end

 initial
  begin
    #10 x=1'b0;
    #25 y=1'b1;
  end

  initial
      #50 $finish;
endmodule
```

```
time            statement executed
0               m = 1'b0;
5               a = 1'b1;
10              x = 1'b0;
30              b = 1'b0;
35              y = 1'b1;
50              $finish;
```


TAIWAN TECH

Structured Procedures: `always` statement

- Start at time 0
- Execute the statements in a looping fashion
- Example

```
module clock_gen;
  reg clock;

  // Initialize clock at time zero
  initial
    clock = 1'b0;

  // Toggle clock every half-cycle (time period =20)
  always
    #10 clock = ~clock;

  initial
    #1000 $finish;
endmodule
```

Can we move this to the always block?

What happens if such a $finish is not included?

TAIWAN TECH

Procedural Assignments

- Assignments inside `initial` and `always`
- Are used to update values of `reg`, `integer`, `real`, or `time` variables
  - The value remains unchanged until another procedural assignment updates it
  - In contrast to continuous assignment (Dataflow Modeling, previous chapter)

TAIWAN TECH

Procedural Assignments (cont'd)

- Syntax
  - *<lvalue> = <expression>*
  - *<lvalue>* can be
    - `reg`, `integer`, `real`, `time`
    - A bit-select of the above (e.g., `addr[0]`)
    - A part-select of the above (e.g., `addr[31:16]`)
    - A concatenation of any of the above
  - *<expression>* is the same as introduced in dataflow modeling
  - What happens if the widths do not match?
    - LHS wider than RHS => RHS is zero-extended
    - RHS wider than LHS => RHS is truncated (Least significant part is kept)

TAIWAN TECH

Procedural Assignments (cont'd)

- The two types of procedural assignments
  - Blocking assignments
  - Non-blocking assignments
- Blocking assignments
  - are executed in order (sequentially)
  - Example:

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count;
initial begin
   x=0; y=1; z=1;
   count=0;
   reg_a= 16'b0; reg_b = reg_a;
   #15 reg_a[2] = 1'b1;
   #10 reg_b[15:13] = {x, y, z};
   count = count + 1;
end
```

All executed at time 0

executed at time 15

All executed at time 25

TAIWAN TECH

Procedural Assignments (cont'd)

- Non-blocking assignments
  - The next statements are not blocked for this one
  - Syntax:
    - *<lvalue>* **<=** *<expression>*
  - Example:

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count;
initial begin
  x=0; y=1; z=1;
  count=0;
  reg_a= 16'b0; reg_b = reg_a;
  reg_a[2] <= #15 1'b1;
  reg_b[15:13] <= #10 {x, y, z};
  count <= count + 1;
end
```

All executed at time 0

Scheduled to run at time 15

Scheduled to run at time 10

TAIWAN TECH

Procedural Assignments (cont'd)

- Application of non-blocking assignments
  - Used to model concurrent data transfers
  - Example: Write behavioral statements to swap values

```
//Emulate the behavior of nonblocking assignments by
//using temporary variables and blocking assignments
always @(posedge clock)
begin
   //Read operation
   //store values of right-hand-side expressions in temporary variables
    temp_a = a;
    temp_b = b;
    //Write operation
    //Assign values of temporary variables to left-hand-side variables
    a = temp_b;
    b = temp_a;
end
```


TAIWAN TECH

Procedural Assignments (cont'd)

- Recommendation
  - Concurrent data transfers => race condition
  - Use non-blocking assignments wherever concurrent data transfers
  - Example: pipeline modeling
  - Disadvantage:
    - Lower simulation performance
    - Higher memory usage in the simulator

TAIWAN TECH

Procedural Assignments (cont'd)

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count;


//All behavioral statements must be inside an initial or always block
initial
begin
       x = 0; y = 1; z = 1; //Scalar assignments
       count = 0; //Assignment to integer variables
       reg_a = 16'b0; reg_b = reg_a; //initialize vectors

       #15 reg_a[2] = 1'b1; //Bit select assignment with delay
       #10 reg_b[15:13] = {x, y, z} //Assign result of concatenation to
                                    // part select of a vector
       count = count + 1; //Assignment to an integer (increment)
end
```


TAIWAN TECH

Event-Based Timing Control

- Events can be utilized to trigger execution of a statement or a block of statements.
  - Usually used to trigger the Clock and Reset
  - Latch vs D-FlipFlop

```
//A level-sensitive latch with asynchronous reset
always @( reset, clock, d)
                              //Wait for reset or clock or d to change
begin
       if (reset)               //if reset signal is high, set q to 0.
              q = 1'b0;
       else    if(clock)        //if clock is high, latch input
              q = d;
end


//A positive edge triggered D flipflop with asynchronous falling
//reset can be modeled as shown below
always @(posedge clk, negedge reset) //Note use of comma operator
if(!reset)
   q <=0;
else
   q <=d;
```


TAIWAN TECH

Behavioral Modeling Statements: Conditional Statements

- Just the same as `if-else` in C
- Syntax:

```
if (<expression>) true_statement;

if (<expression>) true_statement;
else  false_statement;

if (<expression>) true_statement1;
else if (<expression>) true_statement2;
else if (<expression>) true_statement3;
else default_statement;
```

- *True* is `1` or non-zero
- *False* is `0` or ambiguous (`x` or `z`)
- More than one statement: `begin end`

TAIWAN TECH

Conditional Statements (cont'd)

```
//Type 1 statements
if(!lock) buffer = data;
if(enable) out = in;

//Type 2 statements
if (number_queued < MAX_Q_DEPTH)
begin
       data_queue = data;
       number_queued = number_queued + 1;
end
else
       $display("Queue Full. Try again");

//Type 3 statements
//Execute statements based on ALU control signal.
if (alu_control == 0)
       y = x + z;
else if(alu_control == 1)
       y = x - z;
else if(alu_control == 2)
       y = x * z;
else
       $display("Invalid ALU control signal");
```


TAIWAN TECH

Behavioral Modeling Statements: Multiway Branching

- Similar to `switch-case` statement in C
- Syntax:

```
case (<expression>)
   alternative1: statement1;
   alternative2: statement2;
      ...
   default: default_statement; // optional
endcase
```

- Notes:
  - <expression> is compared to the alternatives in the order specified.
  - Default statement is optional

TAIWAN TECH

Multiway Branching (cont'd)

- Examples:

```
//Execute statements based on the ALU control signal
reg [1:0] alu_control;
...
...
case (alu_control)
  2'd0 : y = x + z;
  2'd1 : y = x - z;
  2'd2 : y = x * z;
  default : $display("Invalid ALU control signal");
endcase
```


TAIWAN TECH

Multiway Branching (cont'd)

- Example 2:

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);

// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
reg out;

always @(s1 or s0 or i0 or i1 or i2 or i3)
case ({s1, s0}) //Switch based on concatenation of control signals
      2'd0 : out = i0;
      2'd1 : out = i1;
      2'd2 : out = i2;
      2'd3 : out = i3;
      default: $display("Invalid control signals");
endcase

endmodule
```


TAIWAN TECH

Multiway Branching (cont'd)

- `casex` and `casez` keywords
  - `casez` treats all `z` values as "don't care"
  - `casex` treats all `x` and `z` values as "don't care"
- Example 3:

```
reg [3:0] encoding;
integer state;

casex (encoding) //logic value x represents a don't care bit.
4'b1xxx : next_state = 3;
4'bx1xx : next_state = 2;
4'bxx1x : next_state = 1;
4'bxxx1 : next_state = 0;
default : next_state = 0;
endcase
```


TAIWAN TECH

Behavioral Modeling Statements: Loops

- Loops in Verilog
  - `while, for, repeat, forever`
- The `while` loop syntax:

```
while (<expression>)
    statement;
```

```
//Illustration 1: Increment count from 0 to 127. Exit at count 128.
//Display the count variable.
integer count;

initial
begin
       count = 0;

       while (count < 128) //Execute loop till count is 127.
                           //exit at count 128
       begin
              $display("Count = %d", count);
             count = count + 1;
       end
end
```


TAIWAN TECH

Loops (cont'd)

- The `for` loop
  - Similar to C
  - Syntax:

```
for(init_expr;cond_expr;change_expr)
    statement;
```

```
integer count;

initial
   for ( count=0; count < 128; count = count + 1)
         $display("Count = %d", count);
```


TAIWAN TECH

Loops (cont'd)

- The `forever` loop
  - Syntax:

```
forever
    statement;
```

  - Equivalent to `while(1)` => `clock`

```
//Example 1: Clock generation
//Use forever loop instead of always block
reg clock;

initial
begin
      clock = 1'b0;
      forever #10 clock = ~clock; //Clock with period of 20 units
end
```


TAIWAN TECH

Labs

- Loop stuffs
  - Ram access

- Homework 5
  - Chapter 7 exercises 1-4, 10, 12, 13, 14, 15
  - Due date: Next Sunday (Azar 6th)

TAIWAN TECH

Sequential and Parallel Blocks

- Blocks: used to group multiple statements
- Sequential blocks
  - Keywords: `begin end`
  - Statements are processed in order.
  - A statement is executed only after its preceding one completes.
    - Exception: non-blocking assignments with intra-assignment delays
  - A delay or event is relative to the simulation time when the previous statement completed execution

TAIWAN TECH

Sequential and Parallel Blocks (cont'd)

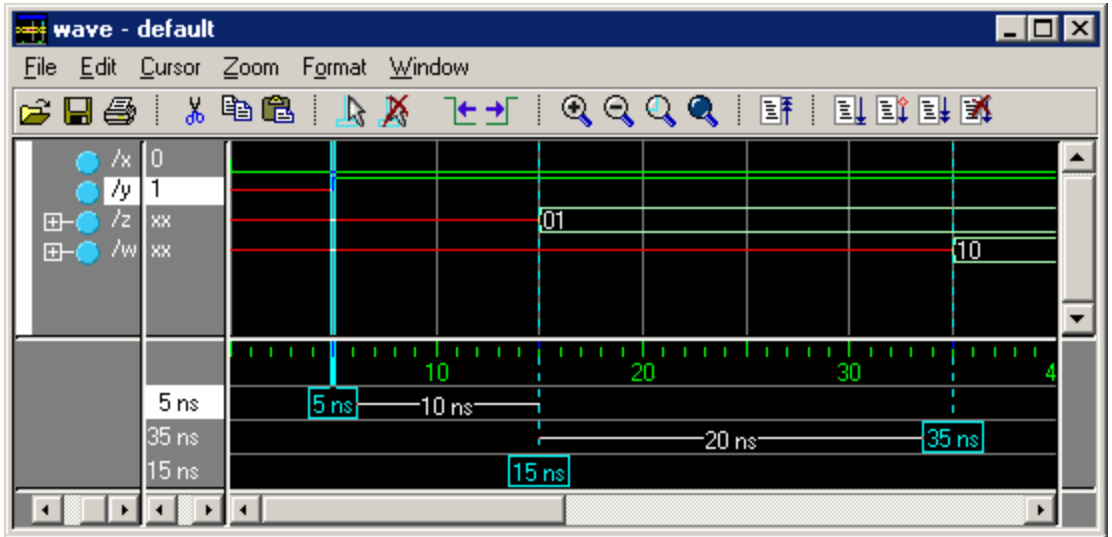
- Parallel Blocks
  - Keywords: `fork, join`
  - Statements in the blocks are executed concurrently
  - Timing controls specify the order of execution of the statements
  - All delays are relative to the time the block was entered
    - The written order of statements is not important

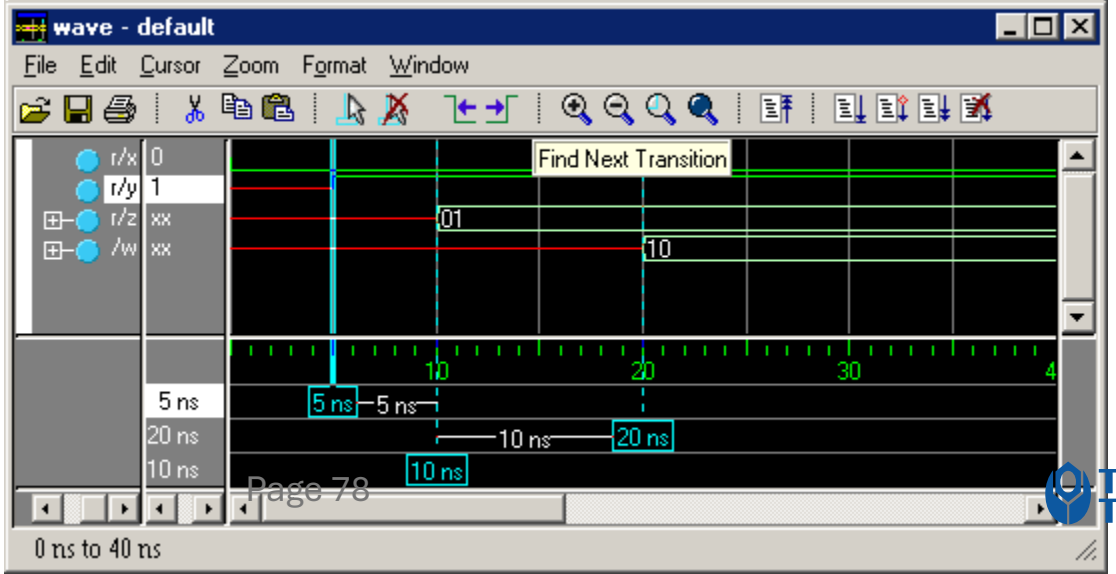
TAIWAN TECH

Sequential and Parallel Blocks (cont'd)

```
initial
begin
    x=1'b0;
    #5    y=1'b1;
    #10   z={x,y};
    #20   w={y,x};
end
```

```
initial
fork
    x=1'b0;
    #5    y=1'b1;
    #10   z={x,y};
    #20   w={y,x};
join
```





TAIWAN TECH

Sequential and Parallel Blocks (cont'd)

- Parallel execution $\Rightarrow$ Race conditions may arise

```
initial
begin
        x=1'b0;
        y=1'b1;
        z={x,y};
        w={y,x};
end
```

- `z, w` can take either `2'b01, 2'b10` or `2'bxx, 2'bxx` depending on simulator

TAIWAN TECH

Special Features of Blocks

- Contents
  - Nested blocks
  - Named blocks
  - Disabling named blocks

TAIWAN TECH

Special Features of Blocks (cont'd)

- Nested blocks
  - Sequential and parallel blocks can be mixed

```
initial
begin
  x=1'b0;
  fork
       #5 y=1'b1;
       #10 z={x,y};
  join
  #20  w={y,x};
end
```


TAIWAN TECH

Special Features of Blocks (cont'd)

- Named blocks
  - Syntax:

```
begin: <the_name>          fork: <the_name>
  …                              …
end                        join
```

  - Advantages:
    - Can have local variables
    - Are part of the design hierarchy.
    - Their local variables can be accessed using hierarchical names
    - Can be disabled

TAIWAN TECH

Special Features of Blocks (cont'd)

```
module top;
    initial
    begin : block1
      integer i;    //hiera. name: top.block1.i
    …
    end

    initial
    fork : block2
    reg i;    //hierarchical name: top.block2.i
    …
    join
endmodule
```


TAIWAN TECH

Behavioral Modeling

Examples

TAIWAN TECH

4-to-1 Multiplexer

```
// 4-to-1 multiplexer. Port list is taken exactly from
// the I/O diagram.
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);

  // Port declarations from the I/O diagram
  output out;
  input i0, i1, i2, i3;
  input s1, s0;
  reg out; //output declared as register

  //recompute the signal out if any input signal changes.
  //All input signals that cause a recomputation of out to
  //occur must go into the always @(...)
  always @(s1 or s0 or i0 or i1 or i2 or i3)
  begin
    case ({s1, s0})
     2'b00: out = i0;
      2'b01: out = i1;
      2'b10: out = i2;
      2'b11: out = i3;
      default: out = 1'bx;
    endcase
  end
endmodule
```


TAIWAN TECH

4-bit Counter

```
//Binary  counter
module counter(Q , clock, clear);

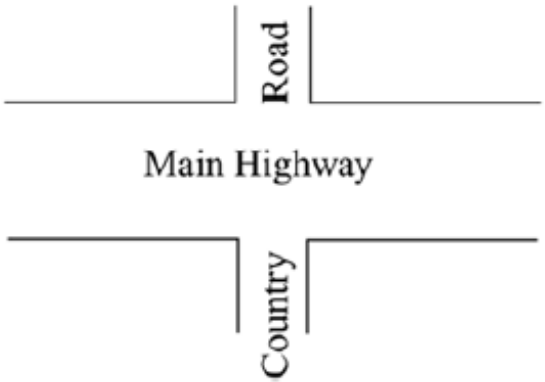
  // I/O ports
  output [3:0] Q;
  input clock, clear;
  //output defined as register
  reg [3:0] Q;

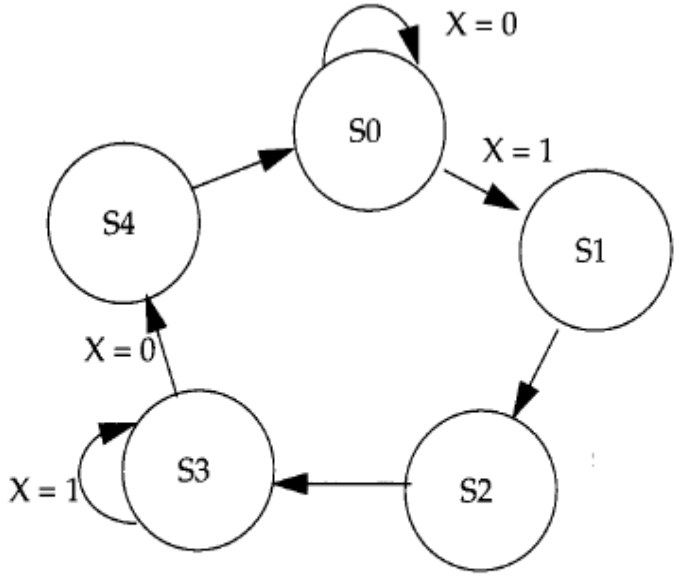
  always @( posedge clear  or negedge clock)
  begin
     if (clear)
       Q = 4'd0;
     else
       //Q = (Q + 1) % 16;
       Q = (Q + 1) ;
  end

endmodule
```


TAIWAN TECH

Traffic Signal Controller





State	Signals
S0	*Hwy = G Cntry = R*
S1	*Hwy = Y Cntry = R*
S2	*Hwy = R Cntry = R*
S3	*Hwy = R Cntry = G*
S4	*Hwy = R Cntry = Y*

TAIWAN TECH

State Maching

```
`define TRUE            1'b1
`define FALSE           1'b0
`define RED             2'd0
`define YELLOW 2'd1
`define GREEN           2'd2

//State definition      HWY         CNTRY
`define S0              3'd0  //GREEN           RED
`define S1              3'd1  //YELLOW          RED
`define S2              3'd2  //RED RED
`define S3              3'd3  //RED GREEN
`define S4              3'd4  //RED YELLOW

//Delays
`define Y2RDELAY        3           //Yellow to red delay
`define R2GDELAY        2           //Red to Green Delay

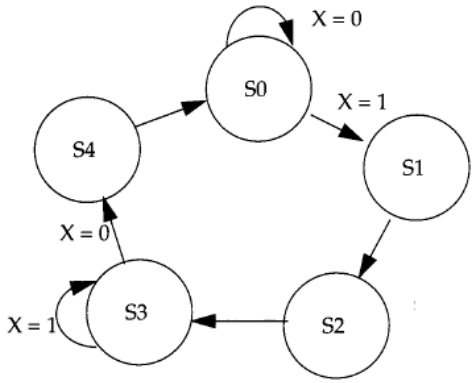
module sig_control (hwy, cntry, X, clock, clear);

//I/O ports
output [1:0] hwy, cntry; //2 bit output for 3 states of signal GREEN, YELLOW, RED;
reg [1:0] hwy, cntry;    //declare output signals are registers

input X;    //if TRUE, indicates that there is car on  the country road, otherwise FALSE

input clock, clear;

//Internal state variables
reg [2:0] state;
reg [2:0] next_state;
```



State	Signals
S0	*Hwy = G Cntry = R*
S1	*Hwy = Y Cntry = R*
S2	*Hwy = R Cntry = R*
S3	*Hwy = R Cntry = G*
S4	*Hwy = R Cntry = Y*

TAIWAN TECH

State Maching

```
//Signal controller starts in S0 state
initial begin
     state = `S0;
     next_state = `S0;
     hwy = `GREEN;
     cntry = `RED;
end

always @(posedge clock) //state changes only at positive edge of clock
     state = next_state;

always @(state) //Compute values of main signal and country signal
begin
     case(state)
             `S0:        begin
                                 hwy = `GREEN;
                                 cntry = `RED;
                         end
             `S1:        begin
                                 hwy = `YELLOW;
                                 cntry = `RED;
                         end
             `S2:        begin
                                 hwy = `RED;
                                 cntry = `RED;
                         end
             `S3:        begin
                                 hwy = `RED;
                                 cntry = `GREEN;
                         end
             `S4:        begin
                                 hwy = `RED;
                                 cntry = `YELLOW;
                         end
     endcase
end
```


TAIWAN TECH

State Maching

```
//State machine using case statements
always @(state or clear or  X)
begin
     if(clear)
            next_state = `S0;
     else
            case (state)
                        `S0: if( X)
                                    next_state = `S1;
                             else
                                    next_state = `S0;
                        `S1: begin //delay some positive edges of clock
                                    repeat(`Y2RDELAY) @(posedge clock) ;
                                    next_state = `S2;
                             end
                        `S2: begin //delay some positive edges of clock
                                    repeat(`R2GDELAY) @(posedge clock)
                                    next_state = `S3;
                             end
                        `S3: if( X)
                                    next_state = `S3;
                             else
                                    next_state = `S4;
                        `S4: begin //delay some positive edges of clock
                                    repeat(`Y2RDELAY) @(posedge clock) ;
                                    next_state = `S0;
                             end
                       default: next_state = `S0;
            endcase
end

endmodule
```


TAIWAN TECH